DATABASE DESIGN DOCUMENT

PART-3: CONSTRAINTS, AUDIT, PERFORMANCE & PARTITIONING

School ERP

PostgreSQL | Spring Data JPA | Hibernate

Prepared by: Database Architecture Team

Document Version: 1.0

Date: 29 July 2026

Preamble

This document is Part-3 of the Database Design Document (DDD) v1.0 for the School ERP System, continuing directly from Part-1: Database Foundation and Part-2: Entity Relationship Standards & Master Data. It defines integrity constraint, audit, soft-delete, partitioning, performance, and archival standards, consolidating Appendix-H Sections 11-15, Appendix-F's Data Integrity/Performance/Scalability/Retention NFR series, and the relevant MDB Part-3 standards into a single physical-design reference.

This document does not regenerate RGD v2.0, any prior Appendix, any UI/UX Screen Specification, or any prior MDB/DDD Part. It contains no SQL, no CREATE TABLE statements, no entity class definitions, no ER/UML diagram, no code, and no examples — every standard is stated as a rule only.

21. Constraint Standards

Consolidates the four constraint types already established across Appendix-H Section 11 and DDD Part-1 Sections 5 and 9 into a single governing reference.

Constraint Type	Enforcement Point	Governing Standard
Primary Key	Database-level, on every table (DDD Part-1 Section 9)	Appendix-H Section 2.3
Foreign Key	Database-level, RESTRICT-only, except the documented polymorphic exception (DDD Part-1 Section 9)	Appendix-H Section 11
Unique Key	Database-level (Section 22 this document)	Appendix-H Section 4, Field 10 per entity
Check Constraint	Database-level for simple range/format rules only (Section 23 this document)	Appendix-G Attribute Catalogue Validation Rule field
Business Rule	Application (Service) layer only — never a database constraint	Appendix-C v1.1; MDB Part-3 Section 24

This table is the definitive answer to "where is this rule enforced" for any new column or table — a rule not clearly assignable to one of these five rows is treated as incompletely specified, not defaulted to any particular layer.

22. Unique Key Standards

Fixes physical implementation of the Unique Constraints already catalogued per entity in Appendix-H Section 4, Field 10.

- Every candidate/business key already identified per entity in Appendix-H (admission_number, invoice_no, barcode, registration_no, etc.) receives a database-level unique constraint, named per DDD Part-1 Section 5 (uq_<table>_<column>).
- Composite unique constraints (e.g., AttendanceRecord's (student_id, timetable_entry_id, attendance_date)) are implemented exactly as already specified per entity in Appendix-H — no composite unique constraint is added or removed at the physical-design stage beyond what Appendix-H already defines.
- A unique constraint that is conditional (e.g., TransportAllocation.student_id unique only where status='Active', per Appendix-H) is implemented as a partial unique index in PostgreSQL, since a standard unique constraint cannot express the conditional clause — this is a physical-design technique, not a change to the logical rule.
- Every unique constraint violation surfaces as the Validation Error response category already fixed in MDB Part-2 Section 15/17, never as an unhandled database exception reaching the client.

23. Check Constraint Standards

Fixes which validation rules from Appendix-G's Attribute Catalogue and Appendix-C v1.1's Business Rules are implemented as database check constraints versus left to the Service layer, extending the Validation Standards already fixed in MDB Part-2 Section 16.

- A check constraint is used only for a rule that depends solely on the values within a single row (e.g., MarksRecord.marks_obtained between 0 and max_marks, LeaveRequest.end_date >= start_date) — this mirrors the Range Validation and simple Cross-Field Validation categories already fixed in MDB Part-2 Section 16.
- A rule requiring a lookup against another table, an aggregate across multiple rows, or external context (e.g., BR-ADM-001 capacity ceiling, BR-FEE-005 waiver ceiling against FeeStructure) is never implemented as a check constraint — it remains exclusively a Service-layer Business Rule (Section 21 this document), since PostgreSQL check constraints cannot reference other tables.
- Every check constraint is named per DDD Part-1 Section 5 (ck_<table>_<rule>) and documents, in its migration record (MDB Part-3 Section 28), which Appendix-G attribute or Appendix-C v1.1 rule it enforces.

24. Audit & History Standards

Fixes the physical realization of the AuditLog entity (Appendix-H ENT-SYS-004) and the mandatory audit-write discipline already fixed in MDB Part-2 Section 20.

- AuditLog is a single, centralized table for the entire schema — no module maintains a parallel, module-specific audit table, consistent with Appendix-I Section 15 and Appendix-J Section 16.
- AuditLog rows are write-once; the physical design provides no application code path or database privilege permitting UPDATE or DELETE against this table under normal operation, consistent with its immutable, write-once nature already fixed in Appendix-H Section 15.
- AuditLog is itself subject to partitioning (Section 26 this document) given its high-volume classification (Section 11, DDD Part-2), keyed by performed_at, consistent with the estimate already fixed in Appendix-H Section 17.7 (15 million+ records/year, estimated).
- Historical Master Data versions (DDD Part-2 Section 15) are a distinct concept from AuditLog — Master Data versioning preserves prior business-meaningful states (e.g., a prior FeeStructure), while AuditLog preserves the change event itself; a table may require both mechanisms simultaneously without redundancy, since they answer different questions ("what was true then" versus "who changed what, when").

25. Soft Delete Standards

Fixes the physical query and indexing implications of the soft-delete baseline already established in DDD Part-1 Section 7 and Appendix-H Section 2.7.

- Every standard read query (Repository layer, MDB Part-3 Section 23) filters is_deleted = false by default — this filter is applied consistently via a shared mechanism (e.g., a Hibernate filter or base repository behavior), not repeated as ad hoc criteria in every individual query method.
- Every index supporting a high-frequency query (Section 27 this document) is evaluated for whether a partial index excluding soft-deleted rows improves performance, given that soft-deleted rows accumulate over time and are rarely part of the active working set.
- A soft-deleted row remains subject to every Foreign Key constraint exactly as an active row would (Section 21) — soft deletion does not relax referential integrity, consistent with Appendix-H Section 11's RESTRICT-only policy.
- Restoration of a soft-deleted record (is_deleted reset to false) is itself an audited action (Section 24), logged as a distinct action type from a standard Update, per the Restore operation already defined in Appendix-G Field 16 (CRUD Operations).

26. Partitioning Standards

Fixes native PostgreSQL declarative partitioning for the high-volume tables already identified in Appendix-H Section 17.8 and Appendix-N Section 15.1/17.8, without altering which tables are partitioned.

Table	Partition Key	Rationale (Already Established)
AttendanceRecord	academic_session_id	10 million+ records/year (Appendix-H Section 17.7); aligns with the academic-session reporting boundary
AuditLog	performed_at (monthly)	15 million+ records/year, estimated; write-heavy, recency-scoped queries (Appendix-N Section 15.1)
NotificationLog	dispatched_at (monthly)	2 million+ records/year, estimated; delivery-status queries are typically recency-scoped
Invoice / Payment	academic_session_id	500,000+ transactions/year; aligns with the same academic-session reporting boundary used school-wide

Partition management (creating new partitions ahead of an approaching academic session boundary, per Section 28 this document) is a scheduled operational task, not an ad hoc administrative action — the specific scheduling mechanism remains Client/Product Decision Required, consistent with MDB Part-4's Background Job standards.

27. Query Performance Standards

Consolidates the indexing and query-pattern standards already fixed in Appendix-H Section 12, DDD Part-1 Section 10, and MDB Part-3 Section 27 into operational guidance for ongoing query design.

- Every new query touching a high-volume table (Section 11 classification, DDD Part-2) is reviewed against an execution plan before merge, as part of the Code Review Checklist already fixed in MDB Part-3 Section 30 — a full-table scan on a high-volume table is treated as a defect, not an acceptable trade-off, at review time.
- Pagination (DDD Part-1/MDB Part-3 Section 23) is mandatory, not optional, for any query capable of returning a high-volume result set — this is enforced at the Repository interface level, not left to caller discipline.
- Lazy-fetch is the default for every JPA relationship (MDB Part-3 Section 27); any eager-fetch exception is documented at the point of declaration with the specific access pattern justifying it.
- Query performance against the ≤200ms indexed-query target (Appendix-F NFR-PERF-012) is measured against production-representative data volume in Staging (Appendix-N Section 4), not against Development-scale data, since index effectiveness at low volume can mask a problem that only appears at scale.

28. Archival & Retention Standards

Fixes the physical mechanism for the archival policy already established per entity in Appendix-H Section 15 and the retention principles in Appendix-F NFR-RET-001/NFR-ARC-001.

- Closed-academic-session data (per the partition boundaries in Section 26) becomes archival-eligible on session close, consistent with Appendix-H Section 15 — archival moves data to lower-cost storage while remaining queryable, it does not delete data.
- Exact retention periods per entity/table remain Client/Product Decision Required, consistent with the same open item already tracked in Appendix-F NFR-RET-001 and Appendix-G Field 19 — this DDD fixes the mechanism (partition-based archival eligibility), not the numeric period.
- Archived partitions remain subject to the identical Foreign Key, constraint, and audit standards already fixed in this document (Sections 21-24) — archival is a storage-tier change, not a relaxation of data-integrity governance.
- Purge (permanent deletion) of archived data requires a logged, IT Admin-approved exception, consistent with Appendix-G Field 21 — no automated purge job runs without this explicit approval step.

29. Data Integrity Verification Standards

Fixes ongoing, post-deployment verification practice, extending the one-time ERD Quality Validation already performed in Appendix-H Section 20 into a recurring operational discipline.

- Referential integrity (no orphaned rows) is verified on a scheduled basis in Production, in addition to the database-level Foreign Key constraints (Section 21) — this is a defense-in-depth check, not a substitute for the constraints themselves.
- Polymorphic association integrity (Section 21's documented exception) is verified by a scheduled check confirming every owner_ref_id resolves to an existing row in the table named by owner_type, since this specific relationship cannot be enforced by a database constraint.
- Duplicate-key and uniqueness verification (Section 22) is monitored via the same mechanism, catching any application-layer bug that might otherwise attempt to bypass a unique constraint through a code path not yet covered by testing (Appendix-M Section 8).
- Verification findings are logged and routed through the same Incident/Change Management process already fixed in MDB Part-4 Section 39 — a data-integrity finding in Production is treated with the same severity discipline as an application defect (Appendix-M Section 14).

30. Database Performance Review Checklist

Verified for every schema change and every release with database impact, extending the Code Review Checklist already fixed in MDB Part-3 Section 30 and the Production Readiness Checklist in MDB Part-4 Section 40.

#	Checklist Item
1	Every new/changed table carries the full Common Columns baseline (DDD Part-1 Section 7) with no exception
2	Every table's classification (Section 11, DDD Part-2) is stated and matches its actual read/write pattern
3	Every Foreign Key, Unique Key, and Check Constraint is present and correctly named (Sections 21-23)
4	High-volume tables use the partition key already fixed in Section 26, with no unpartitioned high-volume table introduced
5	Every new query against a high-volume table is paginated and index-supported (Section 27)
6	Soft-delete filtering (Section 25) is applied consistently, not ad hoc per query
7	AuditLog write path (Section 24) is confirmed present for every state-changing operation on the new/changed table
8	Archival eligibility (Section 28) is defined for any new high-volume/transactional table
9	No Business Rule has been incorrectly implemented as a database check constraint (Section 23)
10	Data Integrity Verification (Section 29) coverage is extended to include the new/changed table